NeuralPathPlanning — Implementation Guide

> This guide replaces the original `Follow_me.pdf` and provides complete, reproducible steps for a new user to set up and run the system from scratch.

Table of Contents

---

1. System Overview

This is a **two-machine** ROS-based autonomous navigation system:

Machine A — Robot PC

Ubuntu 20.04 + ROS Noetic + Gazebo

- Gazebo simulation (Husky robot)
- Depth camera → PointCloud2
- Builds 2-D occupancy map
- `DiffusionPathPlanningL.py` (state-machine navigator)
- `depth_to_pointcloud.py`

task.json + map image →

← **result.json**

SSH / SCP

Machine B — GPU Server

CUDA + PyTorch (no ROS needed)

- Diffusion model inference
- Generates path waypoints from map image
- `neuralPathPlanning.py` daemon
- `neuralPath.py` (PolygonPathGenerator)

Data flow summary: 1. Gazebo publishes depth + RGB camera images. 2. `depth_to_pointcloud.py` converts depth images to `PointCloud2` on `/velodyne_points`. 3. ROS navigation stack (`local_planner`, `terrain_analysis`) uses the point cloud for obstacle avoidance. 4. `DiffusionPathPlanningL.py` builds a 2-D bird's-eye occupancy map and — when a goal is received — sends the map + task to the GPU server via SCP. 5. The GPU server runs diffusion-model inference (`neuralPathPlanning.py`) and writes a `result.json` with world-frame waypoints. 6. Machine A polls for `result.json`, extracts the waypoints, and publishes them to the local planner.

---

2. Prerequisites

Machine A (Robot PC)

Requirement	Version
Ubuntu	20.04 (Focal)
ROS	Noetic
Python	3.8+
Gazebo	11
catkin	any recent

Python packages (Machine A):

```
pip3 install numpy scipy opencv-python-headless
```

ROS packages (installed via `apt` or built from source):

```
ros-noetic-cv-bridge
ros-noetic-sensor-msgs
ros-noetic-nav-msgs
ros-noetic-geometry-msgs
ros-noetic-message-filters
```

Machine B (GPU Server)

Requirement	Version
CUDA	11.8+
Python	3.9+ (virtualenv recommended)
PyTorch	2.x with CUDA support

Python packages (Machine B):

```
pip install torch torchvision diffusers opencv-python pillow numpy
```

> **IMPORTANT:** Machine B does **not** need ROS installed. It communicates via flat files over SSH.

---

3. Repository Structure

```
NeuralPathPlanning/
├── Follow_me.pdf                  ← original (superseded by this guide)
├── src/
│   ├── vehicle_simulator/
│   │   └── launch/
│   │       ├── system_garage.launch   ← main launch file
│   │       └── husky_simulator.launch
│   ├── depth_to_pointcloud/
│   │   ├── CMakeLists.txt
│   │   └── src/scripts/
│   │       ├── depth_to_pointcloud.py      ← depth → PointCloud2 (Machine A)
│   │       ├── DiffusionPathPlanningL.py   ← state-machine navigator (Machine A)
│   │       ├── pointsfilters.py            ← helper filters (Machine A)
│   │       └── GPU_Dir/ros_bridge/
│   │           ├── neuralPathPlanning.py   ← inference daemon (Machine B)
│   │           ├── neuralPath.py           ← PolygonPathGenerator class (Machine B)
│   │           └── mmNet.py               ← neural network definitions (Machine B)
│   ├── local_planner/
│   ├── terrain_analysis/
│   ├── terrain_analysis_ext/
│   ├── sensor_scan_generation/
│   └── husky/
```

---

4. Build the Workspace (Machine A)

This section covers every step needed to compile the ROS workspace from a clean clone.

4.1 Install system-level ROS dependencies

```
# Core ROS Noetic packages needed by this workspace
sudo apt-get update
sudo apt-get install -y \
    ros-noetic-cv-bridge \
    ros-noetic-pcl-ros \
    ros-noetic-sensor-msgs \
    ros-noetic-nav-msgs \
    ros-noetic-geometry-msgs \
    ros-noetic-message-filters \
    ros-noetic-tf2-ros \
    ros-noetic-rviz \
    ros-noetic-gazebo-ros-pkgs \
    ros-noetic-gazebo-ros-control \
    ros-noetic-robot-state-publisher \
    ros-noetic-joint-state-publisher
```

4.2 Install Python dependencies (Machine A)

```
pip3 install numpy scipy opencv-python-headless
```

4.3 Resolve remaining rosdep dependencies

`rosdep` will catch any package-level dependencies declared in `package.xml` files that you may have missed:

```
# One-time initialisation (skip if already done)
sudo rosdep init
rosdep update

# Run from the workspace root
cd ~/NeuralPathPlanning
rosdep install --from-paths src --ignore-src -r -y
```

> **NOTE:** `--ignore-src` skips packages that are already checked out in `src/` so rosdep only installs system packages.

4.4 Build the workspace

```
cd ~/NeuralPathPlanning
catkin_make
```

Expected output (last few lines):

```
[100%] Linking CXX executable /home/<user>/NeuralPathPlanning/devel/lib/local_planner/
localPlanner
[100%] Built target localPlanner
```

If you have a multi-core machine you can speed up the build:

```
catkin_make -j$(nproc)
```

If you only want to rebuild a single package (e.g., after editing `local_planner`):

```
catkin_make --pkg local_planner
```

4.5 Source the workspace

Every terminal that runs ROS nodes from this workspace must source it:

```
source ~/NeuralPathPlanning/devel/setup.bash
```

To make this permanent for all new terminals, add it to `~/.bashrc`:

```
echo "source ~/NeuralPathPlanning/devel/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

4.6 Verify the build

Confirm ROS can find the packages:

```
rospack find local_planner
rospack find vehicle_simulator
rospack find sensor_scan_generation
rospack find terrain_analysis
```

Each command should print the absolute path to the package directory with no errors.

Verify the main launch file is accessible:

```
roslaunch --files vehicle_simulator system_garage.launch
```

This lists all included launch files — a quick sanity check that the launch tree resolves correctly without needing to start Gazebo.

---

5. Machine A — Robot PC Setup

5.1 Create the local temp directory

`DiffusionPathPlanningL.py` uses a local temp folder to stage map images and JSON task files before uploading them to Machine B.

```
mkdir -p ~/ros_tmp
```

> **NOTE:** The default path is `~/ros_tmp`. You can change this in the configuration step below.

5.2 Verify camera intrinsics

`depth_to_pointcloud.py` hard-codes the depth camera intrinsics:

```
camera_intrinsicsrgb = np.array([
    [343.159, 0,       320.5],
    [0,       343.159, 240.5],
    [0,       0,       1   ]
])
```

These are for a **640×480 RealSense-style** camera at the defaults set in the Gazebo URDF. If you use a different camera model or resolution, update `fx`, `fy`, `cx`, `cy` accordingly.

---

6. Machine B — GPU Inference Server Setup

5.1 Copy the GPU_Dir scripts to the server

```
# From Machine A, copy the GPU server scripts
scp -r ~/NeuralPathPlanning/src/depth_to_pointcloud/src/scripts/GPU_Dir/ros_bridge/ \
    <GPU_USER>@<GPU_HOST>:~/ros_bridge/
```

Replace `<GPU_USER>` and `<GPU_HOST>` with your actual SSH credentials.

5.2 Create the shared directory

```
# On Machine B
mkdir -p ~/ros_bridge/shared/history
```

5.3 Place your trained model checkpoint

The inference server expects a `.pth` checkpoint file.
Default expected path on Machine B:

```
~/mmNet/radhaRamanOffice_55.pth
```

If your checkpoint is at a different location, update `CKPT_PATH` in `neuralPathPlanning.py` (see Section 7).

5.4 Checkpoint format

The checkpoint must be a `torch.save()` dict with the following keys:

Key	Contents
`"ema"`	EMA copy of the UNet2DModel state dict
`"encoder"`	ResNetVisualEncoder state dict
`"film"`	FiLMGenerator state dict

This matches the format saved by the training loop in `mmNet.py`:

```
torch.save({
    'model': model.state_dict(),
    'ema':   ema.shadow.state_dict(),
    'encoder': encoder.state_dict(),
    'film':  film_gen.state_dict(),
    'optimizer': optimizer.state_dict()
}, f"radhaRaman_bol_{epoch+1}.pth")
```

7. SSH Passwordless Authentication

The navigator on Machine A uploads files and polls results using `scp` and `ssh`. **Passwordless SSH is required** or the planner will hang.

```
# On Machine A — generate key if you don't have one
ssh-keygen -t ed25519 -C "neural_planner"

# Copy public key to Machine B
ssh-copy-id <GPU_USER>@<GPU_HOST>

# Test — should connect without a password prompt
ssh <GPU_USER>@<GPU_HOST> echo "OK"
```

> **WARNING:** If SSH asks for a password at runtime, `DiffusionPathPlanningL.py` will block indefinitely waiting for `result.json`.

---

8. Configuration — Adapt Paths to Your Environment

All user-specific paths are concentrated in two files.

7.1 `DiffusionPathPlanningL.py` (Machine A)

Locate the `__init__` method and update the **Server Config** block:

```
# ---- Server Config ----
self.remote_user = "YOUR_GPU_SERVER_USERNAME"      # e.g. "john"
self.remote_host = "YOUR_GPU_SERVER_IP"            # e.g. "192.168.1.100"
self.remote_dir  = "/home/YOUR_GPU_SERVER_USERNAME/ros_bridge/shared"
self.local_tmp   = "/home/YOUR_LOCAL_USERNAME/ros_tmp"
```

7.2 `neuralPathPlanning.py` (Machine B)

Update the two configuration constants near the top of the file:

```
# ---------------- CONFIGURATION ----------------
SHARED_DIR = "/home/YOUR_GPU_SERVER_USERNAME/ros_bridge/shared/"
CKPT_PATH  = "/home/YOUR_GPU_SERVER_USERNAME/mmNet/YOUR_CHECKPOINT.pth"
# -----------------------------------------------
```

7.3 Summary table

Variable	File	Location
`remote_user`	`DiffusionPathPlanningL.py`	`__init__` Server Config block
`remote_host`	`DiffusionPathPlanningL.py`	`__init__` Server Config block
`remote_dir`	`DiffusionPathPlanningL.py`	`__init__` Server Config block

Variable	File	Location
`local_tmp`	`DiffusionPathPlanningL.py`	`__init__` Server Config block
`SHARED_DIR`	`neuralPathPlanning.py`	top-level constants
`CKPT_PATH`	`neuralPathPlanning.py`	top-level constants

---

9. Running the Full Pipeline

Open **four separate terminals** on Machine A and one on Machine B.

Step 1 — Machine B: Start the inference daemon

```
# On Machine B (GPU server)
cd ~/ros_bridge
CUDA_VISIBLE_DEVICES=0 python neuralPathPlanning.py
```

> **TIP:** Change `CUDA_VISIBLE_DEVICES=0` to the index of the GPU you want to use (e.g., `1` for the second GPU).

You should see:

```
✔ Loading model from /home/.../mmNet/radhaRamanOffice_55.pth
🚀 Inference daemon started on Desktop-B (Robust Mode)
```

The daemon now polls `shared/task.json` every 50 ms.

Step 2 — Machine A: Launch the ROS simulation

```
# Terminal 1 — source workspace first if not in .bashrc
source ~/NeuralPathPlanning/devel/setup.bash
roslaunch vehicle_simulator system_garage.launch
```

This starts: - Gazebo with the Husky robot in the `P1` world - `local_planner` (obstacle avoidance) - `terrain_analysis` and `terrain_analysis_ext` - `sensor_scan_generation` - RViz for visualization

Wait until Gazebo is fully loaded and you see the robot in RViz before proceeding.

Step 3 — Machine A: Start the depth → point cloud converter

```
# Terminal 2
python3 ~/NeuralPathPlanning/src/depth_to_pointcloud/src/scripts/depth_to_pointcloud.py
```

This node: - Subscribes to `/camera/depth/image_rect_raw` (depth) and `/camera/image` (RGB) - Projects depth pixels into 3-D space using the camera intrinsics - Publishes `PointCloud2` on `/velodyne_points` with XYZI fields (intensity = 0.5 so the local planner registers obstacles)

Step 4 — Machine A: Start the state-machine navigator

```
# Terminal 3
python3 ~/NeuralPathPlanning/src/depth_to_pointcloud/src/scripts/DiffusionPathPlanningL.py
```

This node: - Subscribes to `/registered_scan` (point cloud), `/way_point` (goal), `/state_estimation` (odometry) - Builds a 2-D occupancy map at 1 Hz - When a goal arrives → uploads map image + `task.json` to Machine B, waits for `result.json` - Publishes waypoints on `/wpts` and the dense path visualization on `/radha`

Step 5 — Send a goal

In RViz, use the **2D Nav Goal** tool to click a destination, **or** publish directly:

```
# Terminal 4
rostopic pub /way_point geometry_msgs/PointStamped \
  "header: {frame_id: 'map'} point: {x: 5.0, y: 3.0, z: 0.0}" --once
```

Expected console output (Machine A):

```
[Goal] New goal: (5.0, 3.0)
[Planner] Planning... (goal_id=...)
[Nav] -> NAVIGATING. Waypoint 0/N: (x.xx, y.yy)
🎯 Reached waypoint 0
...
✅ GOAL REACHED. -> IDLE
```

Expected console output (Machine B):

```
📥 Processing Task 1234567890
📤 Sent result for Task 1234567890
```

---

10. ROS Topic Reference

Subscribed Topics

Topic	Type	Node	Description
`/camera/depth/image_rect_raw`	`sensor_msgs/Image`	depth_to_pointcloud	Raw depth image from camera
`/camera/image`	`sensor_msgs/Image`	depth_to_pointcloud	

Topic	Type	Node	Description
			RGB image (for red-object detection)
`/way_pointp`	`sensor_msgs/PointCloud`	depth_to_pointcloud	Optional YOLO obstacle point cloud
`/registered_scan`	`sensor_msgs/PointCloud2`	DiffusionPathPlanningL	Accumulated point cloud from sensor_scan_generation
`/way_point`	`geometry_msgs/PointStamped`	DiffusionPathPlanningL	Navigation goal in map frame
`/state_estimation`	`nav_msgs/Odometry`	DiffusionPathPlanningL	Robot pose estimate

Published Topics

Topic	Type	Node	Description
`/velodyne_points`	`sensor_msgs/PointCloud2`	depth_to_pointcloud	Depth-derived obstacle cloud (XYZI)
`/wpts`	`geometry_msgs/PointStamped`	DiffusionPathPlanningL	Current active waypoint for local planner
`/radha`	`nav_msgs/Path`	DiffusionPathPlanningL	Full dense path (for RViz visualization)

11. Key Parameters

`DiffusionPathPlanningL.py` ROS Parameters

Set via `rosparam` or a launch file:

Parameter	Default	Description
`~grid_resolution`	`0.05`	Meters per pixel in the occupancy map
`~grid_size_pixels`	`500`	Map image size (500×500 pixels = 25m × 25m)
`~vehicle_width`	`0.8`	Husky width in metres (used for obstacle inflation)
`~safety_margin`	`0.0`	Extra inflation beyond vehicle half-width (metres)

Tunable constants (in source)

Constant	File	Default	Description
`waypoint_reach_radius`	`DiffusionPathPlanningL.py`	`0.5 m`	Distance threshold to consider a waypoint reached
`max_points`	`DiffusionPathPlanningL.py`	`50 000`	Max stored map points (prevents memory growth)
`blocked_count` threshold	`DiffusionPathPlanningL.py`	`5`	Consecutive blocked checks before replanning
`intensity_value`	`depth_to_pointcloud.py`	`0.5`	Point cloud intensity (must be > 0.12 for local planner)

`neuralPath.py` / `neuralPathPlanning.py` constants

Parameter	Default	Description
`world_grid_size`	`500`	Must match `grid_size_pixels` on Machine A
`model_grid_size`	`128`	Internal UNet resolution
`resolution`	`0.05`	Must match `grid_resolution` on Machine A
`num_steps`	`20`	Diffusion denoising steps (higher = better quality, slower)

> **IMPORTANT:** `world_grid_size`, `model_grid_size`, and `resolution` **must be identical** on both Machine A (in `DiffusionPathPlanningL.py`) and Machine B (in `neuralPathPlanning.py`). A mismatch causes path coordinates to be wrong.

---

12. Troubleshooting

Robot doesn't move after a goal is sent

1. Check that the inference daemon is running on Machine B and printed `🚀 Inference daemon started`.
2. Verify SSH is passwordless: `ssh <GPU_USER>@<GPU_HOST> echo OK` should return immediately.
3. Check `~/ros_tmp/` on Machine A — you should see `input_<task_id>.png` and `task_<task_id>.json` appear.

4. Check `~/ros_bridge/shared/` on Machine B — you should see these files arrive and `result.json` appear.

"Timeout. Retrying..." message loops

- Machine A did not receive `result.json` within 60 seconds.
- Check network connectivity between the two machines.
- Check that `SHARED_DIR` in `neuralPathPlanning.py` matches `remote_dir` in `DiffusionPathPlanningL.py`.
- Check `CKPT_PATH` — if the model file doesn't exist the daemon will crash silently; watch Machine B's terminal.

`No valid points to publish` from depth_to_pointcloud

- The depth image values may be `NaN` or outside the distance filter range (`0.5 m < x < 6.0 m`).
- Verify the Gazebo camera is publishing on `/camera/depth/image_rect_raw` with `rostopic echo`.
- Confirm the camera is not too close to or too far from obstacles.

Local planner ignores obstacles

- The `intensity` field must be > `0.12` (the default `obstacleHeightThre`).
  The current default is `0.5` — do not set it to `0`.
- Confirm `/velodyne_points` has the correct XYZI binary layout: `x: offset=0, FLOAT32 y: offset=4, FLOAT32 z: offset=8, FLOAT32 intensity: offset=12, FLOAT32`

`TF_REPEATED_DATA` warnings causing navigation instability

- Ensure only one node publishes the `camera_realsense_link_gazebo` frame.
- Check `/tf` with `rosrun tf view_frames` and identify duplicate publishers.

---

13. Architecture Deep-Dive

12.1 Occupancy Map Building

`DiffusionPathPlanningL.py` maintains a rolling 2-D map:

- Point clouds from `/registered_scan` are accumulated in `global_points` (capped at 50 000 points).
- Every second (`timer_build_map`), an OpenCV image is rendered: obstacle points → black pixels on white background.
- The map is morphologically eroded with a circular kernel of radius `vehicle_width / 2` to create a safety buffer.
- A KD-tree (`scipy.cKDTree`) enables fast nearest-neighbour deduplication and collision checks.

12.2 Neural Path Planner (Diffusion Model)

The planner uses a **conditional DDPM (Denoising Diffusion Probabilistic Model)**:

```
Input:  500×500 RGB map image (obstacles=black, free=white, start=green circle, goal=blue
circle)
        ↓ resized to 128×128
Output: 128×128 grayscale path image (bright pixels = path)
```

Architecture components:

Component	Role
`ResNetVisualEncoder`	Encodes the 128×128 map image → 128-dim feature vector using ResNet-18 + SpatialSoftmax
`FiLMGenerator`	Converts the 128-dim encoding → (γ, β) FiLM modulation parameters (32 channels)
`UNet2DModel`	Standard DDPM U-Net; input channels = 3 (map) + 32 (FiLM-modulated noise) → 1 channel path
`DDPMScheduler`	`squaredcos_cap_v2` noise schedule; 20 denoising steps at inference

After inference, `extract_path_128()` traces the bright path pixels from start to goal using a local-mean greedy walk, then scales coordinates from 128-grid → 500-grid → world meters.

12.3 State Machine

```
IDLE ──(goal received)──► PLANNING ──(path received)──► NAVIGATING
  ▲                          ▲                              │
  │                          │                              │
  └──(goal reached)──────────┘◄──(path blocked × 5)─────────┘
```

- **PLANNING**: uploads map, waits for GPU result, commits waypoints.
- **NAVIGATING**: publishes waypoints at 2 Hz, monitors path for blockage via `is_path_blocked()` (KD-tree density check).
- **IDLE**: cleans up state, waits for next goal.

12.4 File-Based IPC Protocol

```
Machine A                          Machine B
─────────                          ─────────
Write input_{id}.png  ──scp──►  shared/input_{id}.png
Write task.json.tmp   ──scp──►  shared/task.json.tmp
                       ──ssh mv──►  shared/task.json   ← atomic rename

                      shared/result.json  ◄── written atomically via .tmp rename
Poll result.json      ◄──scp──  shared/result.json
Validate task_id match
```

The atomic `mv` / `.tmp` rename pattern on both sides prevents the reader from seeing a partially written file.